

A2_Open_Flow_Steady_State

September 1, 2026

1 The Open-Flow Steady State

A cell is not a closed test tube. It continuously receives substrates from the outside — nutrients, signalling molecules, metabolic precursors — and continuously removes or degrades them. This is the **open-flow** setting that applies to virtually every biological system we care about.

This notebook introduces the first key concept: **the steady state of an open-flow system**. It also shows how to simulate a differential equation with Python — something that is reused in every notebook that follows.

1.1 The Open-Flow Model

We track a single molecular species, a substrate S — this could be a metabolite, a second messenger, or any signalling molecule. Three processes act on S :

Process	Biological meaning	Mathematical term
Constant inflow	production / import at a fixed rate	$+a$
Linear removal	passive degradation or export proportional to amount	$-b \cdot S$
Enzymatic conversion	Michaelis-Menten consumption of S	$-v(S)$

The full equation is:

$$\frac{dS}{dt} = a - b \cdot S - \underbrace{\frac{k_{max} S}{K_m + S}}_{v(S)}$$

Reading this aloud: *the rate of change of S equals inflow minus linear loss minus enzymatic consumption.*

At **steady state**, $dS/dt = 0$: the inflow exactly balances all removal processes. The system finds and holds this balance point automatically.

1.2 Code: Model Definition

The function below encodes the equation above. It takes the current time $\mathbf{t}$, the current value of S , and the parameters, and returns the rate of change dS/dt .

```
[3]: import numpy as np
      from scipy.integrate import solve_ivp
      from matplotlib.pyplot import subplots

[4]: def open_flow_model(t, variables, a, b, k_max, K_m):
      """
      Single-variable open-flow model.

      
$$dS/dt = a - b \cdot S - k_{max} \cdot S / (K_m + S)$$


      t          : current time (required by solve_ivp, not used explicitly here)
      variables  : list containing the single state variable S
      a          : constant inflow rate
      b          : first-order removal rate constant
      k_max      : maximum enzymatic rate (Michaelis-Menten)
      K_m        : Michaelis constant
      """
      S = variables[0]

      enzymatic_rate = k_max * S / (K_m + S)

      dS_dt = a - b * S - enzymatic_rate

      return [dS_dt]
```

1.3 Integrating with solve_ivp

To simulate the model over time we use `solve_ivp` from SciPy — *solve initial value problem*. It needs:

1. A handle to the model function
2. The time range (`t_start`, `t_stop`)
3. The initial value of every variable at $t = 0$
4. Any extra parameters the model function requires (passed via `args`)

`solve_ivp` repeatedly calls the model, calculates how variables change at each step, and accumulates the trajectory.

```
[5]: # Parameters
a     = 1.0   # inflow rate
b     = 0.2   # linear removal rate constant
k_max = 1.5   # maximum enzymatic rate
K_m   = 1.0   # Michaelis constant

# Time range
t_start, t_stop = 0, 30

# Initial condition (starting concentration of S)
```

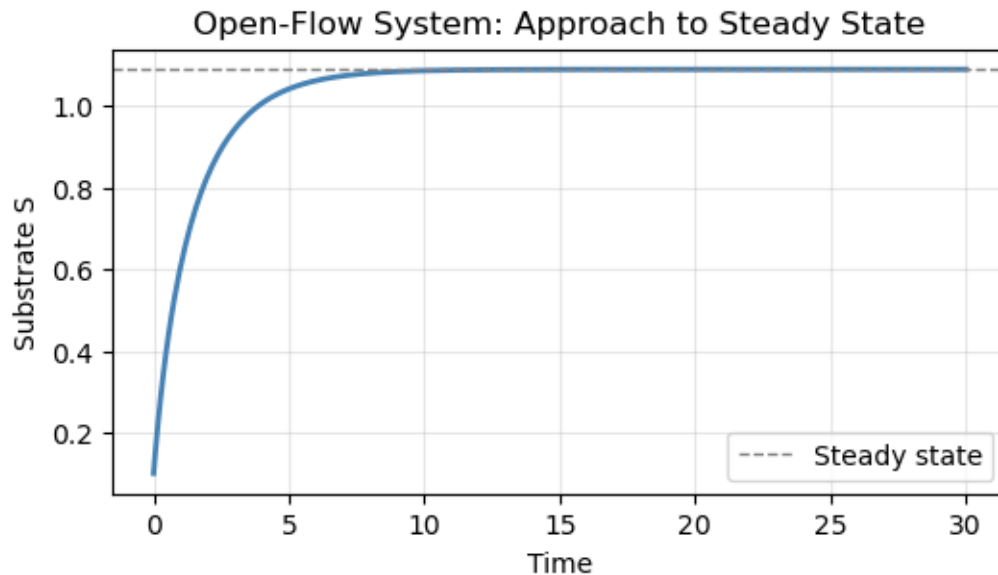
```

S_init = 0.1

# Run the simulation
solution = solve_ivp(
    open_flow_model,
    (t_start, t_stop),
    [S_init],
    args=(a, b, k_max, K_m),
    max_step=0.1
)

# Plot
fig, ax = subplots(figsize=(6, 3))
ax.plot(solution.t, solution.y[0], color='steelblue', linewidth=2)
ax.set_xlabel('Time')
ax.set_ylabel('Substrate S')
ax.set_title('Open-Flow System: Approach to Steady State')
ax.axhline(solution.y[0, -1], color='gray', linestyle='dashed', linewidth=1,
            label='Steady state')
ax.legend()
ax.grid(alpha=0.3);

```



Starting from a low initial concentration, S rises and levels off at a constant value — the steady state.

1.4 The Steady State is Independent of Starting Conditions

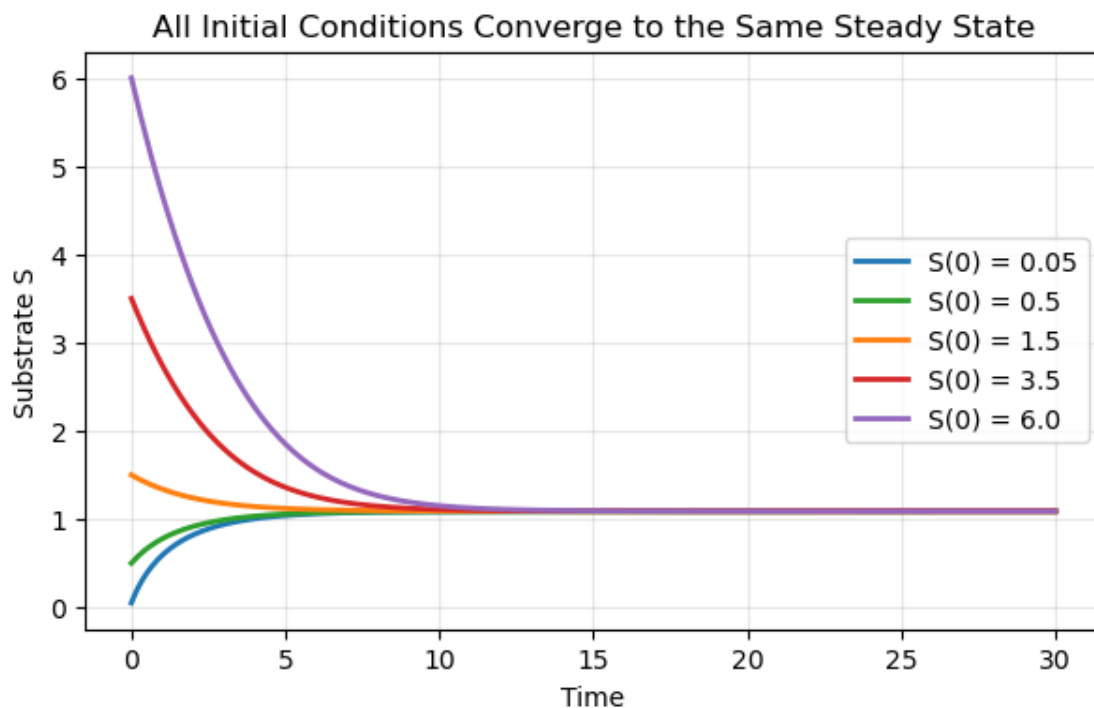
A key property: no matter where you start, the system converges to the same steady state. Below we run the simulation from several different initial conditions.

```
[6]: initial_conditions = [0.05, 0.5, 1.5, 3.5, 6.0]
     colors = ['#1f77b4', '#2ca02c', '#ff7f0e', '#d62728', '#9467bd']

     fig, ax = subplots(figsize=(7, 4))

     for S_init, color in zip(initial_conditions, colors):
         sol = solve_ivp(
             open_flow_model,
             (t_start, t_stop),
             [S_init],
             args=(a, b, k_max, K_m),
             max_step=0.1
         )
         ax.plot(sol.t, sol.y[0], color=color, linewidth=2, label=f'S(0) = {S_init}')

     ax.set_xlabel('Time')
     ax.set_ylabel('Substrate S')
     ax.set_title('All Initial Conditions Converge to the Same Steady State')
     ax.legend(loc='right')
     ax.grid(alpha=0.3);
```



Every trajectory ends at the same horizontal level regardless of where it started. This is **global stability**: the steady state is the unique long-term outcome.

1.5 The State Line and the Steady State

A compact way to visualise the steady state geometrically: plot dS/dt as a function of S . Wherever this curve crosses zero, $dS/dt = 0$ — that is the steady state. Where the curve is positive, S increases; where negative, S decreases. The arrows on the S axis show the direction of motion.

```
[7]: S_vals = np.linspace(0, 8, 400)

dS_dt = a - b * S_vals - k_max * S_vals / (K_m + S_vals)

fig, ax = subplots(figsize=(6, 4))

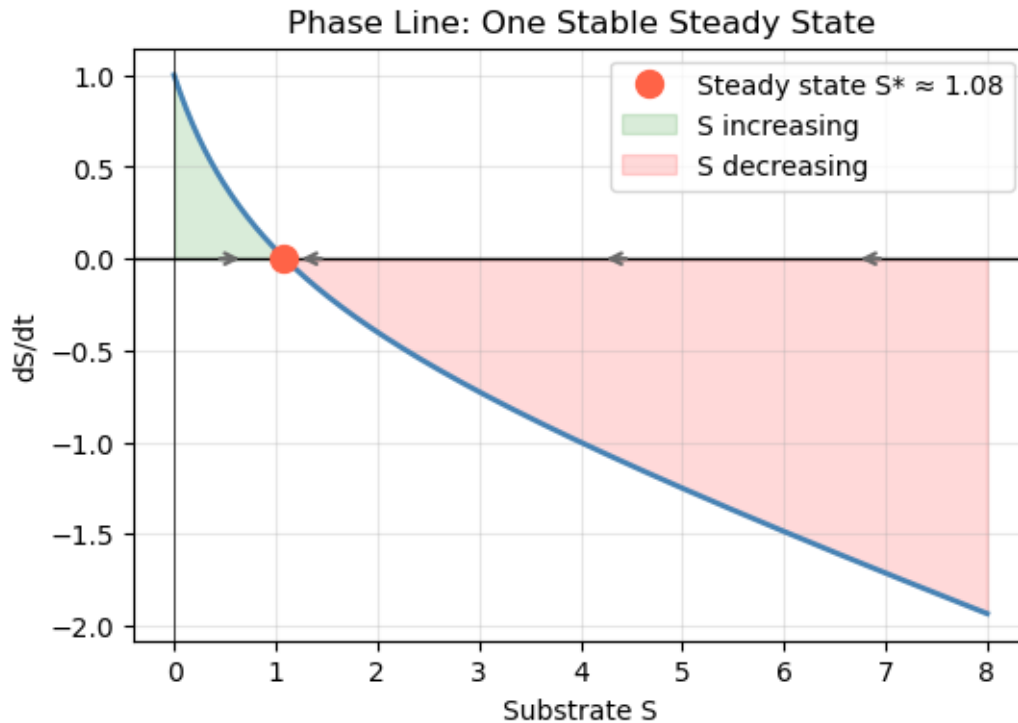
ax.plot(S_vals, dS_dt, color='steelblue', linewidth=2)
ax.axhline(0, color='black', linewidth=1)
ax.axvline(0, color='black', linewidth=0.5)

# Mark the zero-crossing (steady state)
# Find approximate crossing
idx = np.argmin(np.abs(dS_dt))
S_star = S_vals[idx]
ax.plot(S_star, 0, 'o', color='tomato', markersize=10, zorder=5, label=f'Steady_
↪state S*   {S_star:.2f}')

# Direction arrows
for S_arr, sign in [(0.4, 1), (1.5, 1), (4.5, -1), (7.0, -1)]:
    dS = a - b * S_arr - k_max * S_arr / (K_m + S_arr)
    direction = 0.3 if dS > 0 else -0.3
    ax.annotate('', xy=(S_arr + direction, 0), xytext=(S_arr, 0),
                arrowprops=dict(arrowstyle='->', color='dimgray', lw=1.5))

ax.fill_between(S_vals, dS_dt, 0, where=(dS_dt > 0), alpha=0.15, color='green',
↪label='S increasing')
ax.fill_between(S_vals, dS_dt, 0, where=(dS_dt < 0), alpha=0.15, color='red',
↪label='S decreasing')

ax.set_xlabel('Substrate S')
ax.set_ylabel('dS/dt')
ax.set_title('Phase Line: One Stable Steady State')
ax.legend()
ax.grid(alpha=0.3);
```



The curve crosses zero exactly once, and all arrows point towards that crossing: the steady state is stable.

1.6 Changes as a Function of Inflow

Changing the inflow rate a shifts the steady-state level of S . This is physiologically intuitive: a cell receiving more substrate reaches a higher resting concentration.

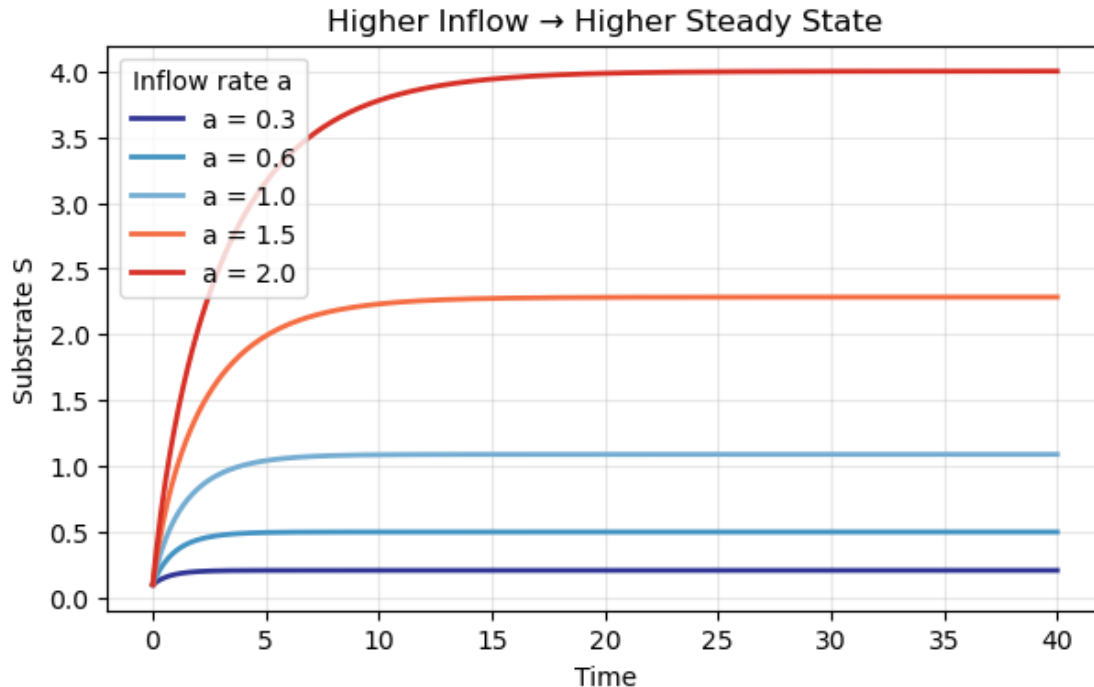
```
[8]: a_values = [0.3, 0.6, 1.0, 1.5, 2.0]
     colors_a = ['#313695', '#4393c3', '#74add1', '#f46d43', '#d73027']

     fig, ax = subplots(figsize=(7, 4))

     for a_val, color in zip(a_values, colors_a):
         sol = solve_ivp(
             open_flow_model,
             (0, 40),
             [0.1],
             args=(a_val, b, k_max, K_m),
             max_step=0.1
         )
         ax.plot(sol.t, sol.y[0], color=color, linewidth=2, label=f'a = {a_val}')

     ax.set_xlabel('Time')
```

```
ax.set_ylabel('Substrate S')
ax.set_title('Higher Inflow → Higher Steady State')
ax.legend(title='Inflow rate a', loc='upper left')
ax.grid(alpha=0.3);
```



1.7 Integrate with odeint

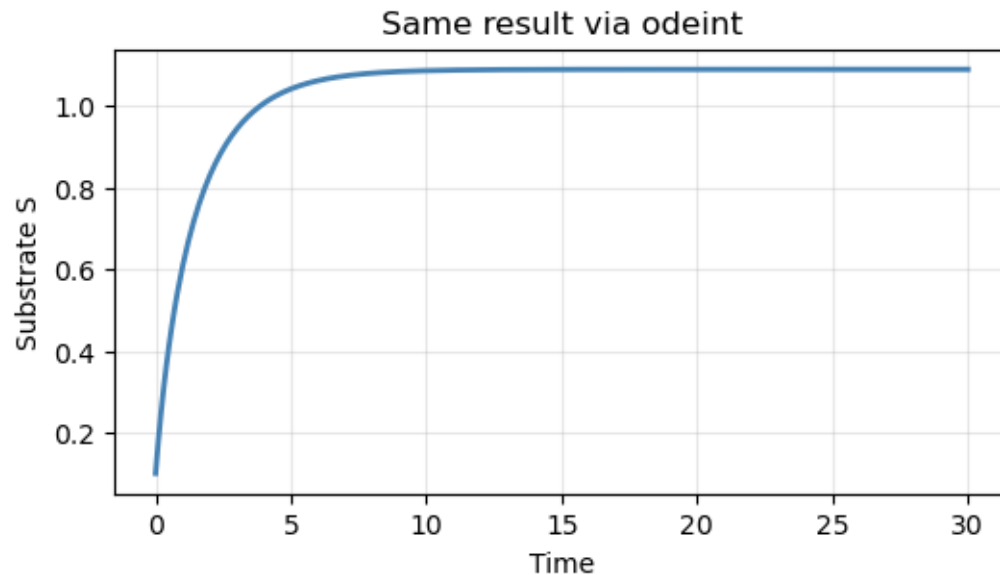
SciPy has a second solver, `odeint`, which you will see used in later notebooks. Its model function takes `(variables, t)` in the opposite order to `solve_ivp`. Otherwise it behaves the same way.

```
[9]: from scipy.integrate import odeint
from numpy import linspace

def open_flow_model_odeint(variables, t, a, b, k_max, K_m):
    """Same model, argument order compatible with odeint (variables first, t_
    ↪second)."""
    S = variables[0]
    enzymatic_rate = k_max * S / (K_m + S)
    dS_dt = a - b * S - enzymatic_rate
    return [dS_dt]

time = linspace(0, 30, 300)
result = odeint(open_flow_model_odeint, [0.1], time, args=(a, b, k_max, K_m))
```

```
fig, ax = subplots(figsize=(6, 3))
ax.plot(time, result[:, 0], color='steelblue', linewidth=2)
ax.set_xlabel('Time')
ax.set_ylabel('Substrate S')
ax.set_title('Same result via odeint')
ax.grid(alpha=0.3);
```



1.8 Summary

- An open-flow ODE captures the biological reality that concentrations are shaped by ongoing inflow and removal.
- When the rate function is a simple MM equation, the system has exactly **one stable steady state**.
- The steady state is where all trajectories end, regardless of initial conditions (the phase line crosses zero once).
- Changing the inflow rate a shifts the steady state continuously.

In **module B** we will replace the MM-Hill term with a different nonlinear rate law — and discover that a single state line can cross zero *more than once*, producing e.g. two stable steady states. That will be called bistability.

2 History: Leibniz on Integration

[]: